MONASH University

Final Report

FIT3164
MDS8

Visual Perception from a Drone's Perspective

Group Member:

Emma Chong Yong Yong
Student ID: 31969763
Email: echo0039@student.monash.edu

Choo Jing Rou
Student ID: 31985246
Email: jcho0103@student.monash.edu

Fong Chun Kent
Student ID: 32189117
Email: cfon0015@student.monash.edu

Project Supervisor:

Professor Raphaël Phan

May 26th, 2023

Word Count : 7832 words

Table of Contents

1. Introduction

In this final report, Monash University's Bachelors of Computer Science in Data Science MDS8 group aims to present and detail the outcomes of our research and development efforts for our final year project, "Visual Perception from a Drone's Perspective". For our project, the deliverable is to develop new advanced models that improve the baseline results of existing source code implementations of human action recognition video analysis techniques, on images/videos taken from aerial view (drone's perspective).

Drone, also known as Unmanned Aerial Vehicle (UAV), is a type of flying robot that can be flown autonomously using software-controlled flight plans. Nowadays, drones are becoming common, and some of them can be bought for less than RM100 from various shopping apps. However, the question of whether we have exploited drones to their fullest potential to enrich our human lives arises. Most drones have cameras attached to or embedded in them, we can now use drones to capture photos and videos that are beyond-human viewpoints. As such, drones are gaining popularity and they are used in various fields for various purposes. For example, drones can be used in rescue operations to search and identify people, or in medical fields for delivery purposes to transport medical supplies and equipment. Hence, our project team is interested in exploring how we could use computer vision techniques to perform video analysis on drone-captured images/videos, in order to perceive the world from the perspective of a drone.

Consequently, our project scope focuses on exploring the various deep learning algorithms and video analysis techniques to develop an enhanced model that focuses on analysing and recognising human actions, from existing open source codes. Furthermore, our project requirements also include evaluating the performance of the models on various real-world settings and datasets, ensuring their practical applicability.

2. Project Background & Literature Review

2.1 Brief Background and Project Rationale

Drawing inspiration from the rapid advances in computer vision and machine learning, vision-based action recognition in video analysis has gained significance as a means to infer human actions (present state) based upon complete action executions (Kong & Fu, 2022). The analysis of human behaviour analysis, such as identification, tracking and analysis of human actions and behaviours, is regarded as an active area of research due to its potential applications across a wide range of fields and domains (Mliki et al., 2020). As a result, this has led to increased attention into exploring the potential of drones in identifying and recognising human activities from the drone-captured videos and images. This is also partly due to the drones' capacity to overcome the issue of fixed coverage because drones have the capacity to capture images from viewpoints beyond that of a normal human, and also the capacity to reach remote or hazardous areas.

While human behaviour analysis is an active area of research interest, fewer research has been done on the development of human action recognition systems based on unmanned aerial vehicles (UAVs), compared to the studies and research on the development of human action recognition systems using stationary cameras (Peng & Razi, 2020). Although there are certain existing commercial drones with gesture recognition capabilities, their capabilities are dependent on high-quality videos captured at low altitudes, and they are restricted to recognising a limited number of simple gestures. Recognising general human actions poses a greater challenge that extends beyond simple gestures, making it a more challenging and complex task for drones (Perera et al., 2019).

The factors that make the human action recognition with UAVs difficult include the difficulties of determining the start and end points of an action, the difficulties and complications of removing the background in the video frames, and the difficulties of dealing with and recognising mixed actions (Peng & Razi, 2020). Therefore, our project team aims to explore computer vision techniques to perform human action analysis on drone-captured videos, and overcome the mentioned challenges in human action recognition with UAVs, in order to perceive the world from the perspective of a drone.

2.2 Literature Review

The objective of our project is to improve the baseline results of the existing open source code implementations of human action recognition video analysis techniques, based on drone-captured videos. Thus, we will review relevant published articles to gain insights and critically evaluate the current work in this field. This report section will provide a short project background introduction, analysis of related articles on drone-captured video analysis, and a conclusion, to provide a better understanding and insight for our project.

2.2.1 Learning Algorithms

Machine learning is part of artificial intelligence, which focuses on enabling computers to perform tasks without explicit programming (Raboaca, 2022). In order to use machine learning to build models, it requires features to be extracted from an input and fed into a classifier to perform classification, which is to find patterns in data and make predictions.

Deep learning is part of machine learning based on artificial neural networks. Different from machine learning, deep learning automatically extract features from data and learn the pattern of it to build the model. CNN, RNN and DNN are some examples of deep learning models. CNN is the most widely used model in classified human behaviour. Appendix Figure 2 shows the deep learning models used to classify video (Belhaouari, 2021). CNNs are a type of deep neural network, consisting of convolutional layers and pooling layers. CNNs were specifically developed for image data analysis tasks, such as handwriting recognition, object identification and localisation, and image content description (Brownlee, 2018). The convolutional layers enable the CNNs to process inputs, such as 2D images or 1D signals, by iteratively reading small segments across the input field, using kernels. At each reading, an internal interpretation of the read input is generated, which will then be projected onto a filter map (Brownlee, 2018). On the other hand, the pooling layers use techniques such as signal averaging or signal maximising to extract the important features from the feature map projections (Brownlee, 2018).

Deep neural network models have recently demonstrated their ability of performing automatic feature learning from raw sensor data, out-performing the hand-crafted domain-specific feature models, and achieving state-of-art results in human activity recognition (Brownlee, 2018). Hence, our base model is utilising deep neural network models, and focusing on frame-by-frame recognition, which detects the human poses and classifies the actions based on the joint analysis within each video frame.

2.2.2 Feature Extraction

Feature extraction is useful and important in classifying human behaviour. Each type of behaviour has their unique features. There are many different types of feature extractions that can be done to extract features from video. In CNN, feature extraction is performed in the convolution and pooling layers. For example, after performing One-dimensional convolution feature extraction algorithm, Two-dimensional convolution feature extraction algorithm will be performed in the convolution layer, and the Feature extraction algorithm for 2D convolution will be performed in the pooling layer (Raboaca, 2022).

For our project, we utilise OpenPose for the feature extraction phase. OpenPose is an open-source library that allows real-time detection of keypoints for the body, feet, hands and face. It consists of three components, which are (1) body and foot detection, (2) hand detection, and (3) face detection. The main component is the combined body and foot keypoint detector, which can also use the original body-only models trained on COCO and MPII datasets (Cao et al., 2019). From the output of the body detector, approximate facial

bounding box proposals can be estimated based on certain body part locations, such as the ears, eyes, nose, and neck (Cao et al., 2019). Similarly, the arm keypoints are used to generate the bounding box proposals for the hands.

2.2.3 How Did We Improve Current Implementations

Current implementations of facial and motion recognition in drone systems heavily rely on pre-trained models. Hence, we will explore the different pre-trained models that can perform action recognition, and which model can improve the accuracy of baseline results.

Firstly, we enhance the current implementations by using the Extreme Gradient Boosting (XGBoost) algorithm proposed by Chen and Guestrin. XGBoost is a scalable end-to-end tree boosting system for learning an ensemble of decision trees (Ayumi, 2016). The technique modifies the boosting process to improve the Taylor expansion of the loss functions. Moreover, XGBoost addresses the issue of imbalance data phenomenon by enabling the selection of the AUC measure for evaluation, and enforcing the proper ordering of imbalance data (Ayumi, 2016). XGBoost has been widely used in various machine learning and data mining applications, demonstrating its ability to achieve state-of-the-art results in various classification tasks (Ayumi, 2016).

Another approach that we have considered to enhance the current implementations is by using Multi Layer Perceptrons (MLP). MLP is a type of neural network model that is composed of interconnected nodal layers arranged in a directed graph structure. Unlike traditional algorithmic computations for feature analysis and classification, MLP is trained rather than programmed (Talukdar & Mehta, 2017). During the training process, the weights of the individual neurons in the MLP are locally adjusted based on their influence on an arbitrary error function. This adaptation of the weight updates not only reduces the learning steps significantly, but it also proves to be computationally efficient in terms of storage and time (Talukdar & Mehta, 2017). Besides that, MLP is also robust against the initial values chosen for the weights.

2.2.5 Related Work

This section includes published articles or research that are relevant to our project of analysing human actions.

According to the article by Peng and Razi (2020), there is an approach in deep learning for video-based action recognition that uses the two-stream method. In this approach, the action is classified using the results of the combination of two distinct convolutional neural networks (CNN) to extract the temporal and spatial features from the video. The spatial CNN is used to model spatial information (action recognition) about the still video frames, and the temporal CNN is used for the recognition of the action from motion in the form of dense optical flow (Simonyan & Zisserman, 2014). In the research done by Simonyan & Zisserman (2014), they experimented using the two-stream methods on two datasets, namely the UCF101 and HMDB51 datasets. The combination of the temporal and spatial CNNs using

averaging was able to achieve an accuracy of 86.9% on the UCF101 dataset, and an accuracy of 58.0% on the HMDB51 dataset. However, the combination of the temporal and spatial CNNs using SVM was able to achieve a higher accuracy of 88.0% on the UCF101 dataset, and also a higher accuracy of 59.4% on the HMDB51 dataset.

Pose-based Convolutional Neural Network (P-CNN) and High-Level Pose Feature (HLPF) descriptors have been used in the evaluation of the dataset we used in this project. The geographical and temporal characteristics of body keypoints throughout the motion are combined to create HLPFs. The 15 keypoints such as head, neck, shoulders, elbows and ankles were used to determine the HLPF. To locate keypoints, Perera et al. (2019) employed the pose estimator OpenPose. P-CNN features were produced utilising body joint locations to extract person-centric motion and appearance elements. The two-stream CNN architecture developed by Simonyan and Zisserman is comparable to the P-CNN methodology. After some modifications and testing, Perera et al. (2019) found that their mean classification accuracy for HLPF was 64.36%, and mean accuracy reported for P-CNN was 75.92%.

Siirtola & Röning (2012) also studied action recognition by using k nearest neighbours (k-nn) and Quadratic Discriminant Analysis (QDA) models. After model building, and training using 5 actions and testing in both offline and online recognition, Siirtola & Röning (2012) found that performance of QDA model was slightly better than k-nn model. While k-nn allows an accuracy of 94.5% in the offline situation, QDA's average classification accuracy is 95.4% depending on the data used to train the models. The average recognition rate when performing online recognition on the device is 95.8% when using QDA, and 93.9% when using k-nn.

2.2.6 Conclusion

In conclusion, unmanned aerial vehicles (UAVs) have significant potential in commercial and security applications. However, there are several challenges that need to be addressed for real-time implementation. These challenges include handling changes in appearance, managing high computational costs, adapting to varying camera views and lighting conditions, and improving the classification rate. It is important to carefully select and implement preprocessing steps to achieve optimal accuracy in action recognition. Additionally, there is a limited availability of aerial video datasets in the UAV domain, with most datasets focusing on object tracking or indoor scenes. This lack of diverse datasets hampers the utilisation of advanced machine learning and deep learning techniques, particularly for analysing human behaviour. Our goal is not only to enhance the existing implementation but also to generate datasets using drones that closely reflect real-world scenarios.

3. Outcomes

3.1 Project Implementation

Our project deliverables include:

1. Developing advanced versions of the baseline existing open source code implementations of human action recognition video analysis technique, as prototypes.
2. The developed prototypes should improve the baseline results.
3. Evaluating the performance of the prototypes on various real-world settings and/or datasets.

To achieve our project deliverables, we made use of two GitHub repositories, which are ChengeYang/Human-Pose-Estimation-Benchmarking-and-Action-Recognition and LZQthePlane/Online-Realtime-Action-Recognition-based-on-OpenPose.

The model used in the ChengeYang GitHub repository is Deep Neural Network (DNN), and we use that model as our base model. We borrowed the existing code in the repository, and implemented three different models based on it. The three models we implemented are namely, the Multi-Layer Perceptron (MLP) model, the Quadratic Discriminant Analysis (QDA) model, and the eXtreme Gradient Boosting (XGBoost) model.

When the users use our project product, they can choose one of the four models, and input a mp4 video file of their interest to analyse the human action into the selected model. Then, the model will output an annotated video with the predicted action(s). More details about the implementation will be provided in Section 4.2 Implementation and Flow of Project.

The implemented project is heavily tested, and all code is kept in consistent syntax and format to ensure users can get the most when using our project. The project is also expected to perform moderately well under certain workload, such that no heavy delay or sudden drop in FPS will occur if minimum hardware requirements are met, as stated in User Guide. With sufficient documentation we believe the project is implemented well for users to navigate. Finally good project management tools and methodologies are also used, such as effective communication and planning using Gantt Charts as schedules management helped a lot in managing the project.

3.2 Achieved Results

After some trial and error, we successfully used the COCO model to retrieve every human action skeleton and joints from each frame. COCO model is a neural-network based approach for detecting and localising key points of human body joints in videos. From the models we built to improve the performance and accuracy, we found that both MLP and XGBoost have better performance than the base model, which is DNN. XGBoost has the best performance among all 4 models. Accuracy table are shown as below:

Model	Accuracy
DNN (Deep Neural Network, Base model)	73.62%
QDA (Quadratic Discriminant Analysis)	63.71%
MLP (Multi-layer Perceptron)	75.77%
XGB (eXtreme Gradient Boost)	83.70%

As mentioned above, the final product of our project is a script which recognises the action being performed by the individual performed in the video. Users can key in their test video directory and the model they want to use, to recognise the human action. The figure below shows the annotated video output and action prediction probability of our model.

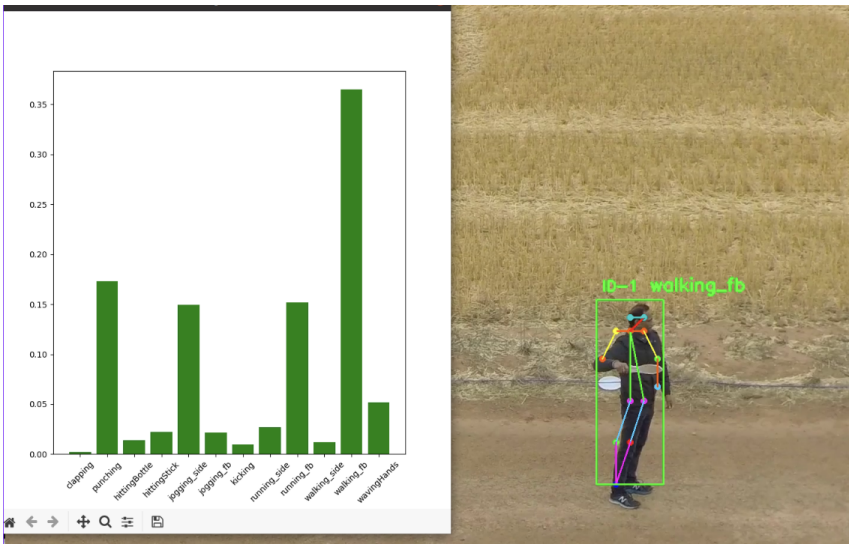


Figure 3.1. Results of Our Project

3.3 Achieved Requirements

Referring to Appendix Figure 1, it shows our Requirements Traceability Matrix, listing the functional and non-functional requirements of our project. There are three functional requirements, and three non-functional requirements. We have achieved all three functional requirements, and one non-functional requirement.

Category	Requirement	How We Achieved It
Functional	Discover various datasets that are applicable to the project brief for recognition purposes to test the baseline results	We found a video dataset online that was created by Perera et al (2019). It contains 240 high-definition video clips, consisting of 66,919 frames that captured 13 human actions from different angles, such as front/back-view and side-view. We split the dataset into

		training and testing datasets, with the ratio of 9:1. We tested and evaluated the performance of the baseline model (DNN) and the three models (MLP, QDA and XGBoost) that we created using the testing dataset. We also recorded a few sample videos of an individual performing some of the listed actions, to test the models. To ensure that our model testing aligns with our project objectives of recognising actions from a drone's perspective, we captured the videos from an aerial view.
Functional	Increase performance and reliability of recognition models on various real-world settings and datasets	As mentioned in the previous requirement, we trained the models using a video dataset that consists of 13 human actions. The actions that our models can recognise include: running fb/side, walking fb/side, jogging fb/side, clapping, waving, punching, kicking, hitting with a bottle, hitting with a stick.
Functional	Implementation of the model on available existing open-source code	We tried to improve the existing base model by changing the structure of the DNN model. In order to obtain higher accuracies, the three models (MLP, QDA and XGBoost) were built on top of the existing code in the ChengeYang GitHub repository. Referring to the performance listed in Section 3.2 Achieved Results, we can observe that the XGBoost model has the best performance among the four models.
Non - Functional	User Profile - to save their own videos analysed	We also analyse the skeleton of every frame in the video and implement a piece of code which helps users to save their own analysed video. Video will automatically be saved in the same directory with the testing video where users' input.

*Note: fb = front/back view, side = side view

3.4 Justification for Our Decisions Made

Decision	Justification
Excluding the stabbing action from the online video dataset	When we were training and testing the models, we decided to exclude the stabbing action after careful consideration that stabbing actions can be associated with violence and harm, and it can also be redundant with the action HittingStick and HittingBottle, since both actions involve similar movements, and the object cannot be detected with ease by our algorithm. 
Model performs action recognition on pre-recorded videos	Previously, we planned to develop models that can perform real-time action recognition. However, due to budget constraints, we could not purchase a drone that allows live-streaming of the captured video to our personal laptops for our models to perform real-time action recognition. Hence, our models can only perform action recognition on pre-recorded videos.
Usage of mobilenet_thin pretrained model over VGG_origin model for pose recognition	While both models are effective to recognise pose framewise, the main idea behind MobileNet is to reduce the computational cost and memory usage of the model while maintaining a good level of accuracy in performing image recognition tasks by using depth wise separable convolution.

3.5 Discussion on Our Achieved Results

This section will include a discussion of the results regarding the performance of four models as well as the performance of the final product.
The confusion matrices below show the performance of the four models by using 10% of our dataset as the testing dataset.

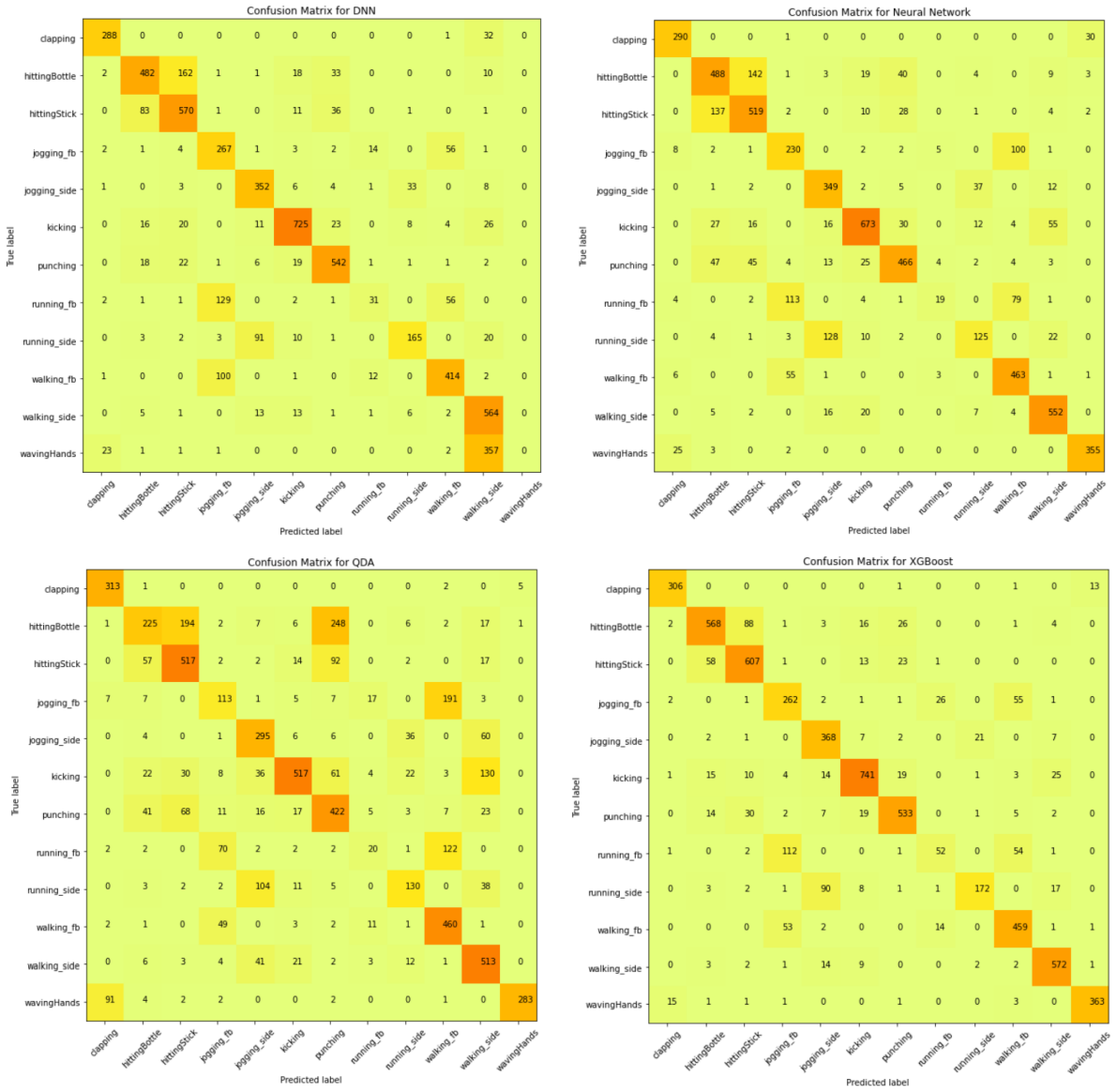


Figure 3.2. Confusion Matrix of the Four Models (DNN, MLP, QDA, XGBoost)

From the confusion matrix above, we can observe that DNN can predict 73.62% of the testing data, but it can't clearly detect running_fb and is confused with jogging_fb and walking_fb. Same goes for the other 3 models that they also can't predict well in running_fb action. This will happen since the skeleton of these 3 actions are similar. But they can detect most of the jogging_fb and walking_fb actions. The DNN model also can't detect wavingHands action and conclude it as walking_side action. Besides, we observed that 4 models can't clearly classify the actions between hittingBottle and hittingStick, although most of the data has been correctly classified. From the big picture, QDA model has the worst performance among 4 models, since it can't perform well in detecting hittingBottle, jogging_fb, running_side and running_fb actions, and having an accuracy of 63.71%. XGBoost model has the best performance among 4 models. It can detect most of the actions and has an accuracy of 83.70%, which surpass the performance of the base model, although this model also can't perform well in detecting runnig_fb action. Hence, we proceed our project by using the XGBoost model.

Since our final product is to detect the action from a given video, we will also discuss the output of the final product. Output video can accurately recognise the position of a human, and the action that the human does. Users can also see the prediction score for every class at each frame. Although there are some times where our model can't recognise well, these instances are only found in a few frames, where the human is changing the action. Other instances are when an action can have overlapping movements, which will result in the same joint placement in the frame. Walking and kicking for example share the same movement, where the leg would be lifted up, resulting in misprediction.

3.6 Limitations of Our Project Outcomes

While our program demonstrates impressive capabilities, there are a few limitations that need to be addressed.

1. Our program is unable to identify multiple human actions within the same video frame.
2. If the input video is overly blurry, the output might exhibit slight deviations in the action positions of the body features, such as the limbs and body, affecting the accuracy of the action recognition.
3. When the body movements in the input video are similar to two or more action classes, the program might struggle to classify the action accurately, as seen with the actions such as Hitting Bottle and Hitting Stick in the confusion matrices shown in Section 3.5 Discussion of Our Achieved Results.
4. There is a limited number of actions that our models can recognise. This is because the models were trained and developed to recognise the 12 aforementioned actions, which are running fb/side, walking fb/side, jogging fb/side, clapping, waving, punching, kicking, hitting with a bottle, hitting with a stick.
5. Our model has been trained and tested using video datasets that feature a single individual in each video frame.

Hence, the performance of our models cannot be guaranteed beyond these specified limitations.

3.7 Discussion of Possible Improvements and Future Works

There are clearly places that can be further enhanced in future works, even though we have accomplished the project goal and all the prerequisites. The possible improvements we can make with our program could be that we could have implemented some form of user interface for users to interact with. For example, users can easily upload their testing file input and get the results by pushing a button "recognise action", rather than put in the one line of long command according to the format. This will make our project more user friendly and easy to use.

Besides, we also can combine our model with a transformer algorithm. Although the pooling layer of the ordinary convolutional neural network (CNN) has image rotation, stretching, and translation invariance, it often doesn't work very well. In the process of action recognition, there are problems such as angle and distortion, which will affect the final recognition result. The Spatial Transformer Networks (STN) network can adaptively spatially change and align data (Cai et al., 2022). To reduce latency and improve efficiency, performing action recognition directly on the drone (edge computing) can be advantageous. This minimises the need for constant data transmission to a central processing unit and enables faster response times. Other improvements include introducing attention mechanisms to focus on relevant spatial or temporal regions within the action sequences to help the network selectively attend to the most informative parts of the input data, improving accuracy and interpretability. Besides, data augmentation can increase the diversity and quantity of training data by applying data augmentation techniques. This can include random cropping, flipping, rotation, scaling, and adding noise or distortions to prevent overfitting.

3.8 Other Matter of Relevance and Interest

Beside human action recognition, we are also interested in implementing animal detection and facial recognition, which are mentioned in our Requirements Traceability Matrix (Requirement IDs 004 and 006). Thus, further improvements should be made to our current model, which we were not able to fulfil due to constraints, mainly time and budget. Besides, other matters of relevance would be related to the environmental factors when a drone performs recognition such as video captured with weather disturbances. These can further be solved by implementing transformers and also other classifiers that detect actions by learning from long-term dependencies, such as LSTM. Lastly, other fields which can be explored include utilising IoT to perform tasks such as agriculture, mining, delivery or even first aid services. In various business sectors, they perform a variety of important tasks (like last mile delivery or mapping) that help companies increase their productivity and the quality of their services and processes (“IoT and Drones”, 2022).

4. Methodology

This section will cover the overall methodology of the project on any deviation from the proposed design in FIT3163, as well as how the design was implemented.

4.1 Overall Design

Since our target users would be students/researchers who are interested in action recognition for drones, our overall design is based on a Command Line Interface (CLI) which is an interface much simpler than a User Interface. To use and run our project, users have to open their terminal (or command prompt) and put commands stated in our user guide. We directly let users key in their drone capture action video path and the model they want to use to recognise human action in accordance with the format we guided in the user guide. The reason for this change is technical users will be more familiar with terminal operation and interface.

```
student@datascience01:~/Desktop/convert_video_csv/Online-Realtime-Action-Recognition-based-on-OpenPose$ python3 main.py --video /home/student/D
ownloads/S10_stabbing_toRight_HD.avi --model /home/student/Desktop/convert_video_csv/Online-Realtime-Action-Recognition-based-on-OpenPose/actio
n_recognition_XGBtestpp.h5
```

Figure 4.1. Example of Command Line With Arguments

For our user interface, we have decided to implement a demonstration video, in addition to our visualisation of our model performance. As opposed to our previous design of our product, which would be a visualisation of how well our model classifies the action based on the probability of each action labels, we also decided to utilise OpenCV to show each frame and this way, users can make side comparisons based on the action labelled during the frame, and the prediction scores for other labels as well. An example of the annotated video output can be seen in Figure 3.1 under Section 3.2 Achieved Results.

Our pre-processing, training and testing phase had been implemented the same way as proposed in FIT3163 with some minor changes in the pre-processing phase where we applied skeleton detection using the COCO model and extracted into a csv file. In the training phase, we also had a minor change in which we changed the ratio of splitting training and testing dataset into 9:1, and we didn't split and use validation dataset.

```
# Normalize the x and y coordinates
for person in data['people']:
  for i in range(0, len(person['pose_keypoints_2d']), 3):
    person['pose_keypoints_2d'][i] /= input_video_frame_width
    person['pose_keypoints_2d'][i+1] /= input_video_frame_height
```

Figure 5 : Dataset Preprocessing Step

```
# train test split
X_train, X_test, Y_train, Y_test = train_test_split(X, dummy_Y, test_size=0.1, random_state=9)

# build keras model
```

Figure 6 : Dataset Splitting Step

4.2 Implementation and Flow of Project

The whole architecture of the project can be mainly divided into 3 sections :

a. Joints Tracking and Processing

When we intend to train our model, we begin by inputting the video file of 12 types of drone captured actions dataset to our OpenPose COCO model. The key innovation of the OpenPose COCO model is its use of multi-stage processing to jointly estimate the locations of all keypoints in a single forward pass of the neural network. Specifically, the model first predicts a heatmap and vectmap for each keypoint, which indicates the likelihood of the keypoint being present at each pixel location in the image. It then uses a technique called PAF (Part Affinity Fields) to estimate the connections between keypoints and generate a set of keypoint pairs representing body parts. With heatmap and vectmap, we can know all the key points in the picture, and then mark the points to everyone through PAFs (Zhang, 2020).

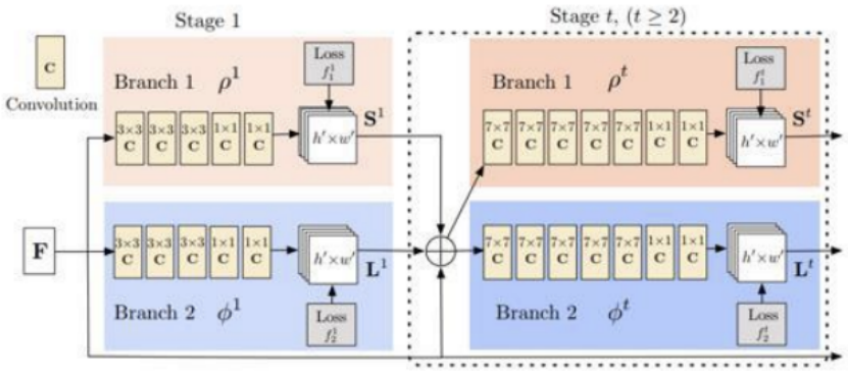


Figure 4.5. Framework of OpenPose COCO Model

The output of this would be 3 values (x, y coordinates and confidence score) of each 36 joints, and the respective action being performed.

AR	AS	AT	AU	AV	AW	AX	AY	AZ	BA	BB	BC
REye_y	REye_c	LEye_x	LEye_y	LEye_c	REar_x	REar_y	REar_c	LEar_x	LEar_y	LEar_c	class
0	0	0.469353645833333	0.436489814814815	0.643376	0	0	0	0.472492708333333	0.444556481481481	0.902129	punching
0.439183333333333	0.0622666	0.469370833333333	0.441970370370370	0.758861	0	0	0	0.4739765625	0.444756481481481	0.874362	punching
0.428427777777778	0.054378	0.470823958333333	0.441885185185185	0.808058	0	0	0	0.475480729166667	0.444732407407407	0.907749	punching
0.441936111111111	0.0859025	0.470883333333333	0.44462037037037	0.824249	0	0	0	0.478541666666667	0.447357407407407	0.887322	punching
0	0	0.470828645833333	0.444573148148148	0.800399	0	0	0	0.475494270833333	0.447374074074074	0.918557	punching
0.433867592592593	0.0907228	0.472353645833333	0.441887037037037	0.725561	0	0	0	0.480042708333333	0.444782407407407	0.879086	punching
0.439255555555556	0.0895669	0.472386979166667	0.441936111111111	0.732005	0	0	0	0.480055208333333	0.447387962962963	0.914204	punching
0.44482962962963	0.207814	0.472421354166667	0.444771296296296	0.890249	0	0	0	0.481620833333333	0.450087962962963	0.961605	punching
0.447261111111111	0.236863	0.472427083333333	0.447338888888889	0.886101	0	0	0	0.483046875	0.450162962962963	0.937832	punching
0.447425	0.154153	0.470928125	0.447398148148148	0.91403	0	0	0	0.4800953125	0.450180555555556	0.91763	punching
0.447365740740741	0.161282	0.470933854166667	0.447437037037037	0.877441	0	0	0	0.4800765625	0.452837037037037	0.906149	punching
0.447477777777778	0.127668	0.4708953125	0.44742962962963	0.858497	0	0	0	0.478529166666667	0.450156481481481	0.875922	punching
0.447531481481481	0.097101	0.470894270833333	0.447294444444444	0.820066	0	0	0	0.4785484375	0.450116666666667	0.918992	punching
0.447384259259259	0.121144	0.470936979166667	0.444785185185185	0.866392	0	0	0	0.478565104166667	0.45007962962963	0.916163	punching
0.447363888888889	0.120387	0.472368229166667	0.4447	0.779963	0	0	0	0.481565104166667	0.447515740740741	0.952551	punching
0.444763888888889	0.162576	0.470925520833333	0.444608333333333	0.80842	0	0	0	0.4800640625	0.447340740740741	0.902173	punching
0.441948148148148	0.193625	0.470903645833333	0.441901851851852	0.870454	0	0	0	0.4800828125	0.444771296296296	0.930833	punching
0.44197962962963	0.116935	0.469326041666667	0.444593518518519	0.849993	0	0	0	0.475453125	0.447331481481481	0.931695	punching

Figure 4.6. Output of Joints Coordinates in CSV Format

We have built and improved 4 action recognition deep learning models, which are DNN, QDA, MLP and XGBoost. We also splitted our dataset into training and testing dataset, which are in the ratio of 90% and 10% respectively.

b. Feature Extraction for Pose Estimation

After we have trained our model, we then proceed with the feature extraction of our video frames for pose estimation. First of all, we applied an object detection algorithm

(YOLO) to detect and localise people in each frame. This step identifies bounding boxes around each person in the frame. Next, we pass this to a backbone model to extract deep visual features from the detected bounding boxes using a pre-trained VGG model/VGG-origin model/Mobilenet model. This step encodes the appearance information of each person. The first few layers of the VGG model are convolutional layers, which apply filters to the input image to extract features such as edges and textures. These layers are followed by a series of pooling layers, which reduce the size of the feature maps and help to make the model more efficient. The final layers of the VGG model are fully connected layers, which use the extracted features to classify the image into one of several categories. These layers are trained using a technique called backpropagation, which adjusts the weights of the model to minimise the difference between the predicted and actual classifications of the images in the training data.

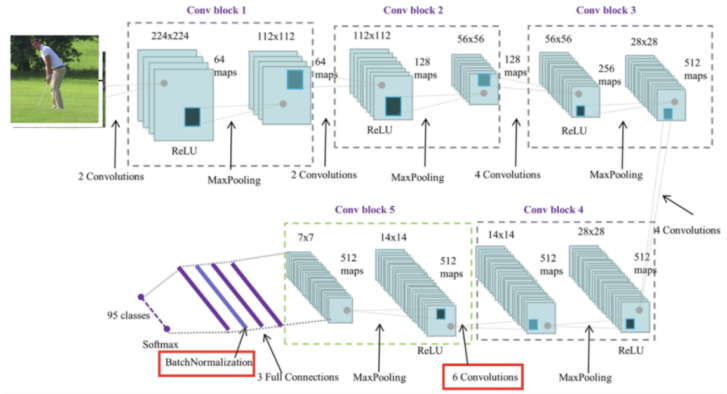


Figure 4.7. Architecture of VGG Origin Model

Next, we utilise DeepSORT (Deep Learning-based SORT) algorithm to associate the pose keypoints and bounding boxes across frames. DeepSORT combines appearance features and motion information to track individuals over time. It assigns unique IDs to individuals and updates the associations based on the similarity between features. Later on, we link the associated pose keypoints with the corresponding tracked individuals using the assigned IDs. This step connects the pose keypoints to specific individuals, enabling pose tracking over time. Finally, we can visualise the tracked poses by overlaying the keypoints on the original frames or use the tracked pose data for further analysis or action recognition tasks.

c. Action Recognition by Pose

Once we obtain the pose keypoints to the specific individual, we perform basic normalisation of the pose, using the coordinates in relation to the video frame, to achieve better results. We can then pass this information as parameters to our trained Action Recognition models.

i) Deep Neural Network (DNN)

An artificial neural network with 4 layers between the input and output is used with the Adam optimiser. Each layer has an activation function, ReLU in our model, which introduces non-linearity to the output before being passed to the next layer, and the last layer performs classification by assigning a probability

score to each action class. The action class with the highest probability is considered the predicted action label for the input video sequence.

ii) Multi Layer Perceptrons

We implemented a fully connected class of feedforward artificial neural network, with hidden layer sizes (20, 30, 40). The layers act similarly with the DNN model.

iii) Quantitative Discriminant Analysis

QDA estimates class-specific mean vectors and covariance matrices based on the training data. QDA uses the estimated covariance matrices and mean vectors to determine the decision boundaries between different action classes. These decision boundaries are based on quadratic equations, which allow for non-linear separations between classes

iv) XGBoost

XGBoost is an ensemble learning algorithm that combines multiple weak learners (decision trees) to create a strong predictive model. It uses gradient boosting to iteratively train and refine the model by minimising the loss function. The parameters for our XGBoost have been fine-tuned, and we have obtained the max_depth = 8 and number of estimators to be 500.

The flow charts of our project for the backend process described above can be seen below.

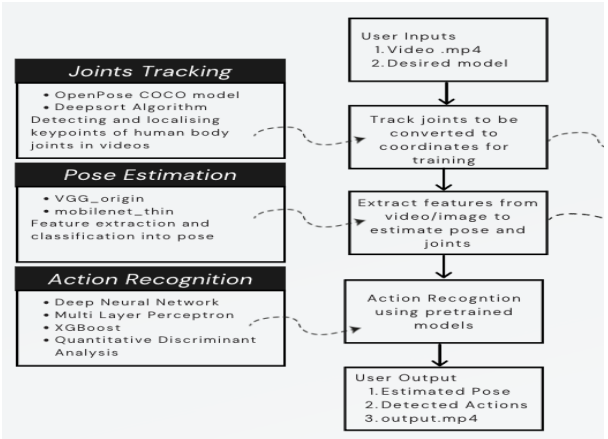


Figure 4.8 : Flow chart of backend architecture

Command line interface wise, the user will first save an action video which is going to be tested in a directory. Then, the human's skeleton will be detected frame by frame with VGG_origin or mobilenet_thin. The desired action recognition algorithm, will provide the predicted labels for each frame. Finally, an analysis video will be shown on the user's screen which contains the detected skeleton with predicted value for each frame.

5. Software Deliverable

5.1 Summary of Software Deliverables

This section documents our project deliverables, with screenshots of sample source code and description of usages. As more detailed descriptions have already been included in the User Guide, this document aims to provide a summary of what is being delivered.

5.1.1 Command Line Interface (CLI)

Our main deliverable is a frontend interface, working on the command line which allows users to upload videos featuring an individual performing specific action(s), and our project will output a new video with bounding boxes and labelled actions. A visualisation of the probability of other predicted classes is shown as well. Users simply need to obtain two parameters :

- --video : File path to the input video. The path should exist and the file extension should be included as well. For example, “punching.mp4”, instead of “punching”
- --model : File path to the saved trained model for action recognition. There is no default value for this parameter, therefore a model must be used. The file extension should be included.

and then run the simple command to call the main.py file script.

```
while cv.waitKey(1) < 0:
    has_frame, show = cap.read()
    if has_frame:
        fps_count += 1
        frame_count += 1

# pose estimation
        humans = estimator.inference(show)
# get pose info
        pose = TfPoseVisualizer.draw_pose_rgb(show, humans)  # return frame,
# recognize the action framewise
        show, data = framewise_recognize(pose, action_classifier)
        Action = list(data.keys())
        Probability = list(data.values())
        bars = ax.bar(Action, Probability, color='green')
        for bar, height in zip(bars, Probability):
            bar.set_height(height)
        plt.pause(5)
        plt.draw()
        height, width = show.shape[:2]
```

Figure 5.1. Sample Source Code for Output of CLI

The code above in Figure 5.1 demonstrates the usage of OpenCV and Matplotlib from Python to obtain frames from the input video, and then passing it to our pose and action recognition model. After obtaining the pose and action labels, bounding boxes are drawn and the probabilities are used to plot a real-time updating bar chart with each frame. An example of the output can be seen in Figure 3.1 under Section 3.2 Achieved Results.

5.1.2 Videos to Joints Coordinates Source Code

The code in Figure 5.2 shows the process of preprocessing video files to csv files, by recognising the joint coordinates in (x,y) confidence scores. This code uses the pretrained COCO model of OpenPose, and will output joint coordinates which will then be passed to our training model for training and testing.

```
import os
from os.path import exists, join, basename, splitext

videos = []
for video in os.listdir(input_directory):
  InputVideo = input_data + video
  folder = output_directory
  !mkdir {folder}
  videos.append(video)
  print(video)

  #!cd openpose && ./build/examples/openpose/openpose.bin --video {InputVideo} --write_json {folder} --display 0 --render_pose 0
  !cd openpose && ./build/examples/openpose/openpose.bin --video {InputVideo} --write_json {folder} --display 0 --render_pose 0 --model_pose COCO

print("Number processed of videos: ", videos.__len__())
```

Figure 5.2. Source Code for Preparing CSV Joint File

The parameters shown in Figure 5.2 indicates :

- --video : file path to input video
- --write_json : folder to be written to
- --display : whether to show frames or not
- --render pose : whether to render each pose or not
- --model_pose : pretrained model to be used.

Users should change the runtime type to “GPU”, as shown in Figure 5.3, and configure the file paths accordingly, as shown in Figure 5.4.

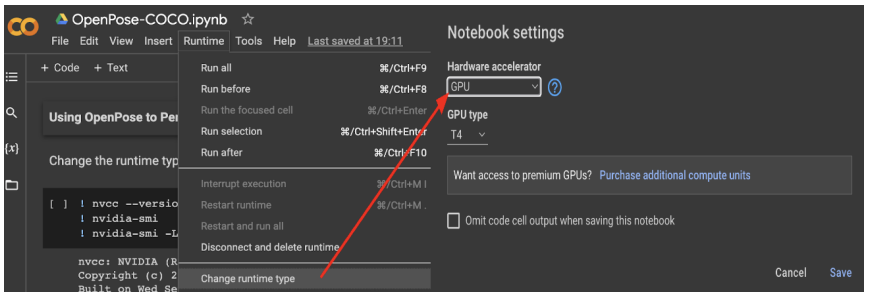


Figure 5.3. Runtime Configuration

```
input_directory = "/content/drive/MyDrive/HittingStick/hitting_stick"
input_data = '/content/drive/MyDrive/HittingStick/hitting_stick/'
output_directory = '/content/drive/MyDrive/HittingStick/hitting_stick_json/'
```

Figure 5.4. File Path Configuration

5.1.3 Action Recognition Model Source Code and Results Matrices

By pipelining our pose recognition algorithm with action recognition algorithms, we can use the joints estimated in the video frames to predict the action class being performed by the user input. We have successfully implemented four types of machine learning/deep learning models, namely DNN, QDA, MLP and lastly XGBoost. By comparing their accuracies and correctly predicted labels, we conclude that XGBoost gives us the best classification accuracy. Figure 5.5 shows an example source code of the base DNN model, while the source codes for the other models will be included in the Appendix section.

```
# build keras model
model = Sequential()
model.add(Dense(units=128, activation='relu'))
model.add(BatchNormalization())
model.add(Dense(units=64, activation='relu'))
model.add(BatchNormalization())
model.add(Dense(units=16, activation='relu'))
model.add(BatchNormalization())
model.add(Dense(units=5, activation='softmax'))  # units = nums of classes
```

Figure 5.5. Sample Source Code for DNN Model

In Figure 5.5, our base model implements a DNN model with 4 input layers, and activation function ReLU. The model will then be used for action recognition after obtaining the joints detected, by using the pose recognition model, in Section 5.1.4 Pose Recognition Model Source Code. The accuracy of the model is also calculated to see how well each model performs. Users can then select the model with the highest accuracy for their project, when implementing on their own. The confusion matrix of the four models can be viewed in the Appendix section.

5.1.4 Pose Recognition Model Source Code

```
def load_pretrain_model(model):
    dyn_graph_path = {
        'VGG_origin': str(file_path / "Pose/graph_models/VGG_origin/graph_opt.pb"),
        'mobilenet_thin': str(file_path / "Pose/graph_models/mobilenet_thin/graph_opt.pb")
                }
    graph_path = dyn_graph_path[model]
    print(graph_path)
    if not os.path.isfile(graph_path):
        raise Exception('Graph file doesn\'t exist, path=%s' % graph_path)

    return TfPoseVisualizer(graph_path, target_size=(input_width, input_height))
```

Figure 5.6. Function to Load Pretrained Pose Models

The pose recognition model is downloaded, and TfPoseVisualizer will be called to read the pretrain model as graph objects to be used. In Figure 5.6, there are two models, mainly VGG_origin and mobilenet_thin which are ready to use. In OpenPose, navigate to the directory /Pose/graph_models/VGG_origin, and run the command line download.sh.

5.2 Software Qualities

5.2.1 Robustness

Our project and models implemented are highly compatible in terms of robustness, such that we have implemented multiple exceptions as well as error handling, and also fully tested with unit testing. Any wrong parameters being input by the users will be captured and be processed to prompt for a new input, instead of stopping the program. Libraries and other dependencies should be installed prior to using our project, as described in the User Guide.
Rating : Excellent

5.2.2 Security

Due to the nature of our initiative, neither the gathering of personal information nor the sharing of confidential information is necessary. Therefore, there won't be any user data leakage. Additionally, the software's output will be saved locally on the end user's device, and its predictions will be deleted after it has finished running. Additionally, because the software only accepts video files, harmful files are not permitted.

5.2.3 Usability

The CLI is easy to use and only involves two parameters, which users can navigate with ease. Norman's 7 Principles is used as well, where erroneous messages are displayed and prompt for new input from users. It is also consistent, as only specific formats can be used. The design concept of constraining refers to determining ways of restricting the kind of user interaction that can take place at a given moment (Norman, 2013).
Rating : Excellent

5.2.4 Scalability

Since our final product would be a command line interface, we wouldn't need to test the scalability element because it wouldn't require any additional workload from numerous concurrent users or storage. There is no requirement for that to be scaled because users can re-upload any videos since the videos used for the input would be saved again in the working directory. After the project closes, there is no longer a requirement for ongoing maintenance.

5.2.5 Documentation & Maintainability

Comments in the source code lines and User Guide are helpful for new users to understand our project, as well as technical users who wish to further continue the implementation of the project. Meaningful variable names are used as well. Unfortunately, the underlying pretrained models, (OpenPose) and Tensorflow packages aren't maintained by us, thus users should always keep the latest version to check for major updates.
Rating : Good

6. Critical Discussion on the Software Project

This section will discuss the evolution of our project, from project proposal until the final product was produced. The project has changed significantly compared to our project proposal.

While doing some research and repository finding, we found that most of the research papers and repositories are outdated and incomplete. Hence, we need to find newer research papers and repositories in order to proceed with our project. Finally, we choose ChengeYang/Human-Pose-Estimation-Benchmarking-and-Action-Recognition and LZQthePlane/Online-Realtime-Action-Recognition-based-on-OpenPose as our main study repositories. But we still need to update some prerequisites in order to use these 2 repositories. In ChengeYang GitHub repository, they only recognise 5 actions. In order to let our model recognise more actions, we choose another dataset which contains 13 classes of actions such as walking, kicking, waving and clapping.

In the feature extraction phase, we choose the COCO model to extract the human skeleton from each frame and record each position of the joint. COCO model is a neural network-based approach for detecting and localising key points of human body joints in videos. Different from the preprocessing and feature extraction steps we have done, SURF is a low-complexity image feature detection method that is an accelerated version of the scale-invariant feature transform (SIFT) to extract local image descriptors. Lucas-Kanade optical flow to all pixels of each frame to find the mapping between keypoints among the consecutive frames, and to quantify their relative motions. Lastly, motion estimation and trajectory smoothing are used to smooth the picture of video (Peng & Razi, 2020). Compared to the COCO model, the COCO model is easy to use and helps us directly extract features from datasets without doing any video stabilisation. Environment factors such as colour, shadows and sundries also can be ignored using COCO model to extract features. This was proven in our blackbox testing phase. Hence, some preprocessing techniques such as Histogram Equalization (HE), Self Quotient Image (SQI), Locally Tuned Inverse Sine Nonlinear (LTISN), Gamma Intensity Correction (GIC), and Difference of Gaussian (DoG) which handle shadows, illuminations and also grey scales of images can be ignored in these steps. Deepsort algorithm is then incorporated into the joints tracked, to draw bounding boxes and frames, as well as detecting the human in the image and drawing lines that connect to show the pose outline of the human.

In our proposal, we plan to use time-series machine learning algorithms such as CNN and RNN in model implementation, improvement and training. RNN is a class of artificial neural networks where connections between nodes can create a cycle, hence it can do action recognition with time changes using multiple frames of human skeleton data. But in ChengeYang GitHub repository, they use Deep Neural Network (DNN), which is not a time-series model since they study about real-time action recognition. Hence, we also let real-time action recognition as our target and use that DNN model as our base model. In order to build real-time action recognition, we borrowed the existing code in the repository,

and implemented three different models based on it. The three models we implemented are namely, the Multi-Layer Perceptron (MLP) model, the Quadratic Discriminant Analysis (QDA) model, and the eXtreme Gradient Boosting (XGBoost) model. These models can predict and classify actions using 1 frame of human skeleton data. Firstly, we built and successfully tested our models using our webcam. While we plan to buy drones which are suitable for us to do real-time action recognition, we found the equipment required is over budget. Hence, we changed our project from real-time action recognition to video-based action recognition. This means we have to change the testing output script, but our models remain the same since these 4 models can be used in both types of action recognition. We choose mobilenet_thin over VGG_origin model to extract features from each frame in our final product. Although both are OpenPose pretrained models for pose estimation, mobilenet_thin uses lesser time and lesser complexity to extract features than VGG_origin. This has been tested in our performance testing phase.

Our team believes that the Agile approach is more suitable for our project, because major changes of direction and scopes are widely common, we can sync with any real time changes that may arise during the product's life cycle, or even beyond it when future implementation is required, by using the Agile approach's flexibility. In order to use the Agile approach in our project, we have captured all details with extensive documentation. Apart from that, meetings and sprint workflows are also frequently scheduled to update our progress.

Finally, after successful implementation of the project, we have a wide range of test cases to test that our project is working as intended. Whitebox or unit testing which tested the internal functions of the project, such as data splitting steps, model prediction classes and data loading functions are made sure it works, and prompts users for correct input. We use blackbox testing to test the boundary values of input videos for our deep learning model. For this test, we input videos with varying dimensions, sizes and colour filters into our deep learning model and tested them manually. There are some limitations while we are going through the testing process. Many modules and libraries are required in this project, since many of the functions rely on certain libraries and modules to function.

Despite all of the challenges that were thrown at the team, we believe that we still produced a product that surpassed our expectations. We not only discovered various datasets that are applicable to the project brief for recognition purposes to test baseline results, we also implemented and increased performance and reliability of recognition models on various real world settings and datasets. Our final product also can let users save their own video which has been analysed. But due to time limitations and lack of knowledge in the relevant field, we didn't implement our model which can detect emotional status based on facial recognition and other additional modifications. Although we didn't implement some non-functional requirements, the resulting program created by the team works efficiently and produces results that were better than we were expecting.

7. Conclusion

To conclude, all of the software deliverables made were fulfilled after adjustments to our initial project proposal. This report aims to cover aspects such as limitations, requirements, scopes and most importantly the accomplishments. We were also able to demonstrate a clear understanding of our software methodology by showing the flow of our project as a whole, as well as management methods.

We were able to increase the accuracy of our DNN base model which performs action recognition on drone-captured dataset, as well as delivering software deliverables such as data preprocessing source codes and trained models which are ready for usage. Nevertheless, there still exist limitations and setbacks, which were discussed in detail, and also any improvements which could have been made.

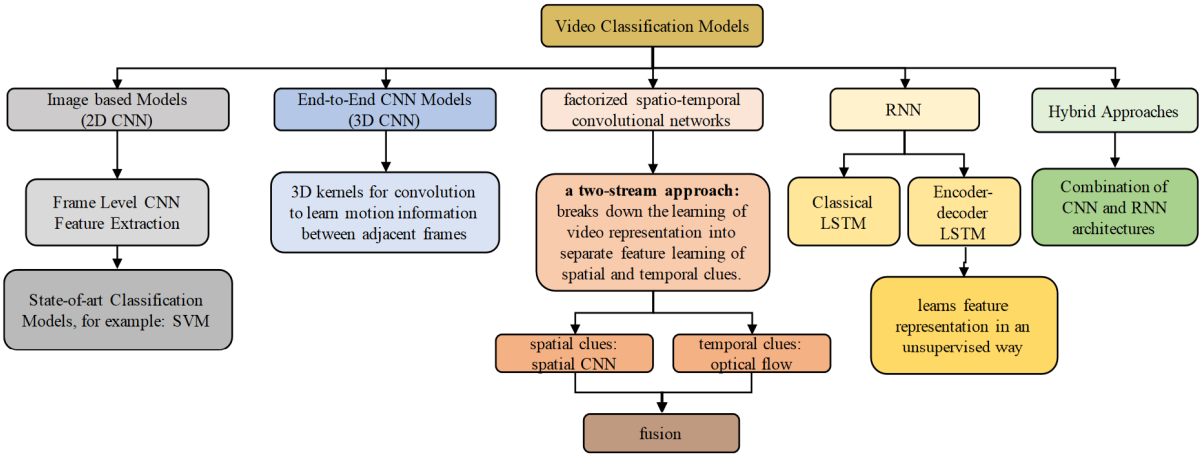
The User Guide as well as Test Report generated serves an important purpose to users so that it acts as a demonstration and also clarity for first time users and also technical users.

To sum it up, we have managed to produce satisfactory outcomes, such that we fulfilled our project objectives and scopes, while overcoming obstacles as a team and also applying software methodology tools and knowledge learnt to the fullest. We have gained immaculate experience from this project, and hope that our findings in Human Action Recognition by UAV Drones can benefit existing and future implementations, particularly in light of evaluating Human Action Recognition.

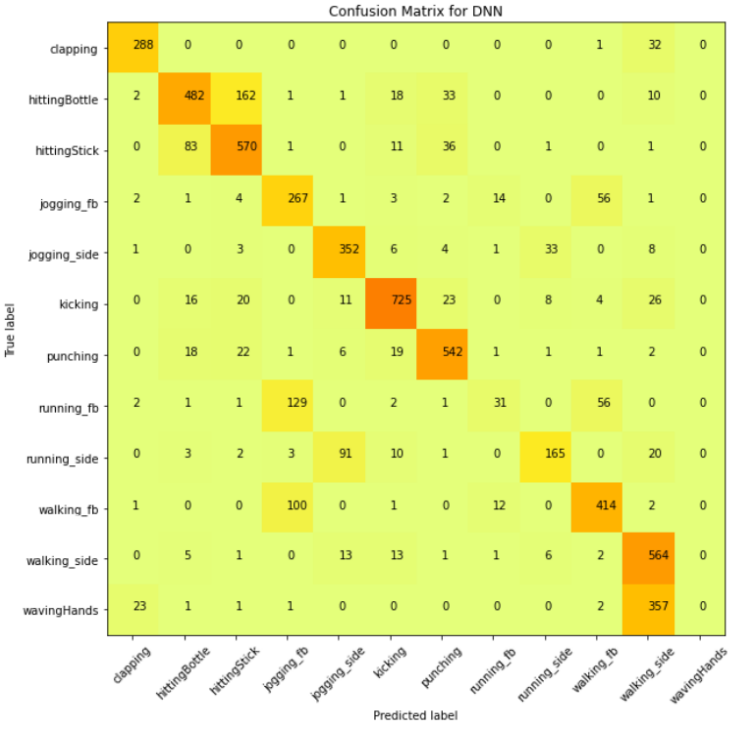
8. Appendix

Requirement ID	***Requirements (Functional or Non-Functional)***	***Assumption(s) and/or Customer Need(s)***	***Category***	***Source***
001	Discover various datasets that are applicable to the project brief for recognition purposes to test baseline results	Sufficient resources and datasets are required to develop required recognition models	Functional Requirement	Client (quoted from Project Brief)
002	Increase performance and reliability of recognition models on various real world settings and datasets	Sufficient resources and datasets are required to develop and improve required recognition models. Sufficient testing to ensure the models are bug-free and crash-free.	Functional Requirement	Client (quoted from Project Brief)
003	Implementation of the model on available existing open-source code	Assuming that the existing open-source code is fully functional without errors	Functional Requirement	Client (quoted from Project Brief)
004	Detect the emotional status based on facial recognition	Sufficient resources and datasets are required to develop recognition models	Non-Functional Requirement	Client (Discussed with Project Team)
005	User Profile - to save their own videos analysed	Assuming the user is relatively well-versed with our programming language of choice and cloud computing, so the usability is secondary	Non-Functional Requirement	Client (Discussed with Project Team)
006	The developed model allows additional modification and customization on other analysis / recognition, such as animal classification	The developed model is a simple model that can be enhanced in the future to provide further capabilities / functionalities	Non-Functional Requirement	Client (Discussed with Project Team)

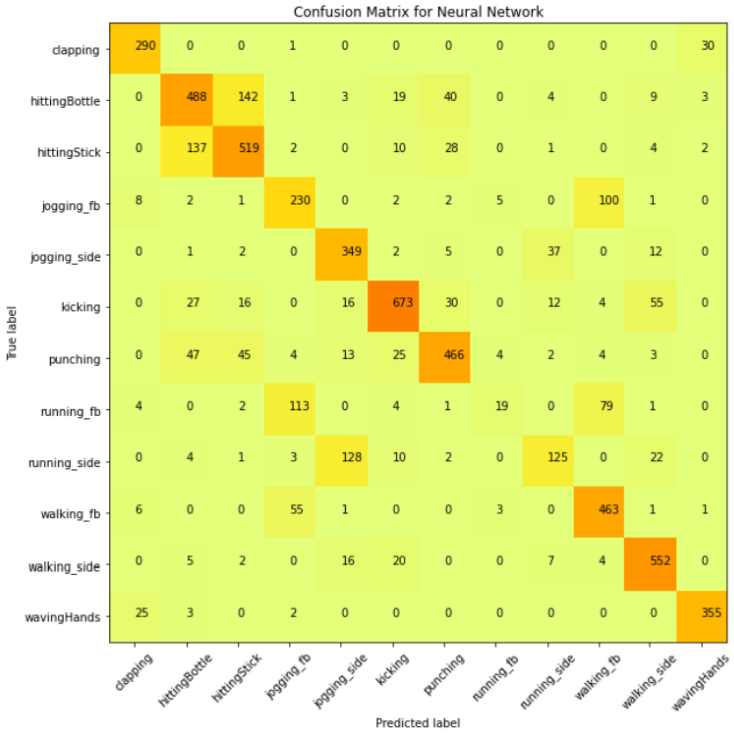
Appendix Figure 1. Requirements Traceability Matrix



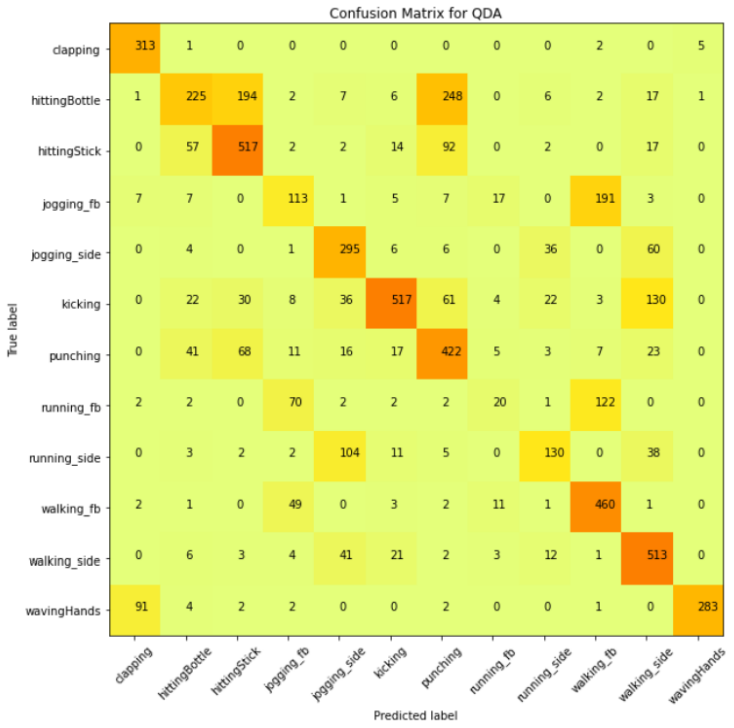
Appendix Figure 2. Summary of Video Classification Approaches (Belhaouari, 2021)



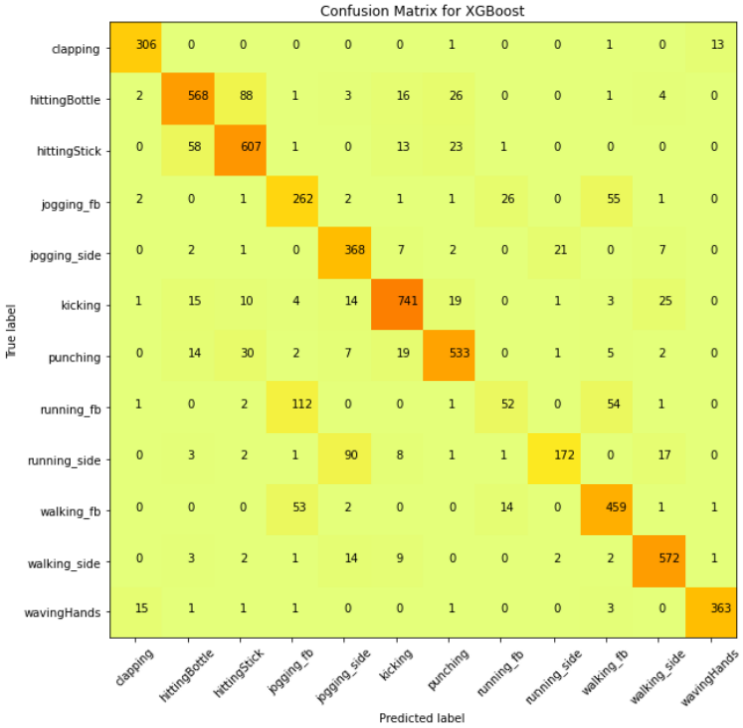
Appendix Figure 3. Confusion Matrix of the DNN Model



Appendix Figure 4. Confusion Matrix of the MLP Model



Appendix Figure 5. Confusion Matrix of the QDA Model



Appendix Figure 6. Confusion Matrix of the XGBoost Model

```python
def framewise_recognize(pose, pretrained_model):
    frame, joints, bboxes, xcenter = pose[0], pose[1], pose[2], pose[3]
    joints_norm_per_frame = np.array(pose[-1])
    data={}

    if bboxes:
        bboxes = np.array(bboxes)
        features = encoder(frame, bboxes)

        # score to 1.0 here).
        detections = [Detection(bbox, 1.0, feature) for bbox, feature in zip(bboxes, features)]

        # implementation of bounding box
        boxes = np.array([d.tlwh for d in detections])
        scores = np.array([d.confidence for d in detections])
        indices = preprocessing.non_max_suppression(boxes, nms_max_overlap, scores)
        detections = [detections[i] for i in indices]

        tracker.predict()
        tracker.update(detections)

        trk_result = []
        for trk in tracker.tracks:
            if not trk.is_confirmed() or trk.time_since_update > 1:
                continue
```

Appendix Figure 8 : Function framewise recognize which predicts actions

```
    while cv.waitKey(1) < 0:
        has_frame, show = cap.read()
        if has_frame:
            fps_count += 1
            frame_count += 1

    # pose estimation
            humans = estimator.inference(show)
    # get pose info
            pose = TfPoseVisualizer.draw_pose_rgb(show, humans)  # return frame, joints, bboxes,
    # recognize the action framewise
            show, data = framewise_recognize(pose, action_classifier)
            Action = list(data.keys())
            Probability = list(data.values())
            bars = ax.bar(Action, Probability, color='green')
            for bar, height in zip(bars, Probability):
                bar.set_height(height)
            plt.pause(5)
            plt.draw()
            height, width = show.shape[:2]
```

Appendix Figure 9 : main.py which calls functions

```
for i in range(len(X)):
    X_pp.append(dpp.pose_normalization(X[i]))
X_pp = np.array(X_pp)

# Encoder the class label to number
# Converts a class vector (integers) to binary class matrix
encoder = LabelEncoder()
encoder_Y = encoder.fit_transform(Y)
matrix_Y = np_utils.to_categorical(encoder_Y)
X_train, X_test, Y_train, Y_test = train_test_split(X_pp, encoder_Y, test_size=0.1, random_state=42)

# Build XGBoost model and train
classification=xgb.XGBClassifier(max_depth=8, learning_rate=0.015, n_estimators=500, objective='multi:
classification.fit(X_train, Y_train, verbose=True)

# preds = classification.predict(X_test)

# accuracy = (preds == Y_test).sum().astype(float) / len(preds)*100

# print("XGBoost's prediction accuracy WITH optimal hyperparameters is: %3.2f" % (accuracy))

# Save the trained model
# classification.save_model('/home/student/Desktop/BaseCode/Human-Pose-Estimation-Benchmarking-and-Act
classification.save_model('action_recognition_XGBtestpp2.h5')
```

Appendix Figure 10 : Implementation of XGBoost Model

```
def load_pretrain_model(model):
    dyn_graph_path = {
        'VGG_origin': str(file_path / "Pose/graph_models/VGG_origin/graph_opt.pb"),
        'mobilenet_thin': str(file_path / "Pose/graph_models/mobilenet_thin/graph_opt.pb")
                }
    graph_path = dyn_graph_path[model]
    print(graph_path)
    if not os.path.isfile(graph_path):
        raise Exception('Graph file doesn\'t exist, path=%s' % graph_path)

    return TfPoseVisualizer(graph_path, target_size=(input_width, input_height))
```

Appendix Figure 11 : Loading of pretrained OpenPose models

```
        parser = argparse.ArgumentParser(description='Action Recognition by OpenPose')
        parser.add_argument('--video', help='Path to video file.')
        parser.add_argument('--model')
        args = parser.parse_args()

# loading models
        estimator = load_pretrain_model('mobilenet_thin')
        action_classifier = xgb.XGBClassifier()
        action_classifier.load_model(args.model)
        # action_classifier = load_model(args.model)


# FPS calculator
        realtime_fps = '0.0000'
        start_time = time.time()
        fps_interval = 1
        fps_count = 0
        run_timer = 0
        frame_count = 0

# webcam mode
        cap = choose_run_mode(args)
        video_writer = set_video_writer(cap, write_fps=int(7.0))
        fig, ax = plt.subplots()
```

Appendix Figure 12 : Helper function called to load pretrain models

```
def createDataFrames(files, data):
  dataframe = pd.DataFrame(data, columns = ["Nose_x", "Nose_y", "Nose_c", "Neck_x", "Neck_y", "Neck_c", "RShoulder_x", "RShoulder_y", "RShoulder_c", "RElbow

  # remove columns ending with "_c" (we don't need the confidence scores)
  dataframe = dataframe.loc[:, ~dataframe.columns.str.endswith("_c")]

  # append the action name of each frame
  dataframe = dataframe.assign(action = files)

  # assign 0 to NaN
  dataframe.fillna(0)
  return dataframe

body_keypoints_df = createDataFrames(fileName,joints)

csv_output = '/content/drive/MyDrive/HittingStick/hitting_stick.csv'
body_keypoints_df.to_csv (csv_output, index = False, header = True)
```

Appendix Figure 13 : JSON file to CSV for joints coordinates output

```
def choose_run_mode(args):
    """
    video or webcam
    """
    global out_file_path
    if args.video:
        # Open the video file
        if not os.path.isfile(args.video):
            print("Input video file ", args.video, " doesn't exist")
            sys.exit(1)
        cap = cv.VideoCapture(args.video)
        out_file_path = str(out_file_path / (args.video[:-4] + '_tf_out.mp4'))
    else:
        # Webcam input
        cap = cv.VideoCapture(0)
        # 设置摄像头像素值
        cap.set(cv.CAP_PROP_FRAME_WIDTH, cam_width)
        cap.set(cv.CAP_PROP_FRAME_HEIGHT, cam_height)
        out_file_path = str(out_file_path / 'webcam_tf_out.mp4')

    return cap
```

Appendix Figure 14 : Interpreting run mode and saving of output

9. References

Ayumi, V. (2016). Pose-based human action recognition with extreme gradient boosting. *2016 IEEE Student Conference on Research and Development (SCOReD)*, 1–5. https://doi.org/10.1109/scored.2016.7810099

Bobick, A. F., & Davis, J. W. (2001). The recognition of human movement using temporal templates. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 23*(3), 257–267. https://doi.org/10.1109/34.910878

Brownlee, J. (2018). *Deep Learning Models for Human Activity Recognition*. Machine Learning Mastery. https://machinelearningmastery.com/deep-learning-models-for-human-activity-recognition/

Cai, Y., Yu, H., Fan, X., Hou, Y., & Zhang, Q. (2022). Human Action Recognition Based on Improved FCN Framework. *2022 8th International Conference on Virtual Reality (ICVR)*, 432–437. https://doi.org/10.1109/icvr55215.2022.9848369

Cao, Z., Hidalgo, G., Simon, T., Wei, S.-E., & Sheikh, Y. (2019). OpenPose: Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 43*(1), 172–186. https://doi.org/10.1109/tpami.2019.2929257

IoT and Drones: Their Capacities and Key Use Cases. Cogniteq. (2022). https://www.cogniteq.com/blog/iot-and-drones-their-capacities-and-role-business

Kong, Y., & Fu, Y. (2022). Human action recognition and prediction: A survey. *International Journal of Computer Vision, 130*, 1366–1401. https://doi.org/10.1007/s11263-022-01594-9

Mliki, H., Bouhlel, F., & Hammami, M. (2020). Human activity recognition from UAV-captured video sequences. *Pattern Recognition, 100*, 107140. https://doi.org/10.1016/j.patcog.2019.107140

Norman, D. (2013, July 8). Don Normans Principles of Design. Design Principles FTW. https://www.designprinciplesftw.com/collections/don-normans-principles-of-design

Peng, H., & Razi, A. (2020). Fully Autonomous UAV-Based Action Recognition System Using Aerial Imagery. Advances in Visual Computing, 12509, 276–290. https://doi.org/10.1007/978-3-030-64556-4_22

Perera, A. G., Law, Y. W., & Chahl, J. (2019). Drone-Action: An Outdoor Recorded Drone Video Dataset for Action Recognition. *Drones, 3*(4), 82. https://doi.org/10.3390/drones3040082

Raghavan, J., & Ahmadi, M. (2021). A modified CNN-based face Recognition System. *International Journal of Artificial Intelligence & Applications, 12*(2), 1–20. https://doi.org/10.5121/ijaia.2021.12201

Siirtola, P., & Röning, J. (2012). User-independent human activity recognition using a mobile phone: Offline recognition vs. real-time on device recognition. *Advances in Intelligent and Soft Computing*, 617–627. https://doi.org/10.1007/978-3-642-28765-7_75

Talukdar, J., & Mehta, B. (2017). Human action recognition system using good features and multilayer Perceptron Network. *2017 International Conference on Communication and Signal Processing (ICCSP)*, 0317–0323. https://doi.org/10.1109/iccsp.2017.8286369

Zhang, T. (2020). Gesture recognition based on Deep Learning. *Journal of Physics: Conference Series, 1449*, 012081. https://doi.org/10.1088/1742-6596/1449/1/012081

10. Team Member's Contribution Annex

Contributed Section	Contributor
Front Cover Sheet	Emma
Introduction	Emma, Jing Rou
Background	Emma
Outcomes	Emma, Jing Rou, Kent
Methodology	Jing Rou, Kent
Software Deliverables	Kent
Critical Discussion on the Software Project	Jing Rou
Conclusion	Emma, Jing Rou, Kent
Appendix	Emma, Jing Rou, Kent
References	Emma, Jing Rou